

Java Core:

What is SOLID Principal?

SOLID principle is an acronym for Object Oriented Design:

- S - Single Responsibility: 1 class should only be responsible for 1 job
- O - Open/Close: an entity should be open for extension but closed for modification:
for short: adding new feature without changing the existing code
- L: Liskov Substitution: child class should work anywhere where its parent is expected to
- I: Interface Segregation: Clients should not be forced to use the interfaces that they don't need
for short: Instead of making 1 large interface, do smaller ones but be specific
- D: Dependency Inversion: High - level component should not depend on low level component.

Both should depend on Abstraction. Abstraction should not depend on details. Details should depend on abstraction.

- + High-level component: Business logic
 - + Low level component: implementation
 - + Abstraction: Interface
- Meaning the interaction between the layer should be through the interfaces

What is OOP? Explain Inheritance, Encapsulation, Polymorphism, Abstraction with example

OOP stands for Object Oriented Programming. It has 4 pillars:

- Encapsulation: The idea of bundling the fields and methods together into 1 class as 1 working unit. There are 4 levels of access modifier to this: Public, Private, Protected, Default
- Abstraction: Only show the essential attributes and hide the unnecessary information such as the implementation of the method
- Inheritance: The fields and method from 1 class can be inherited to another
- Polymorphism: referring to overriding and overloading

What is Is-a and Has-a relationship in Java?

The easiest way that I think about this is that:

- child class/ subclass has an 'is-a' relationship to its parent
- parent classes 'has-a' relationship references to its child class

What is method overriding and method overloading? Provide your own example

- Overriding is referring to the methods being inherited from the parent classes. It will have the same name, same return type, same parameter, and number of parameter but behave differently

- Overloading is referring to the multiple method in 1 class have the same name, same return type but different parameter, and/or number of parameter

Difference between interface and abstract class

- Abstract class is used to provide common base that share codes and states
- Interface defines a contract that any classes using it needs to implement the method body
- A class can implement from multiple interfaces but can only extend from 1 abstract class

What is diamond problem? How can we solve diamond problem?

It is easier for me to explain in terms of an example. The diamond problem is when you have a class let's call A. Then two other classes B and C are extended from it. Then another class D is extended from both B and C. This creates a diamond shape; hence the name of the problem is diamond problem. The problem is class D confuses and does not know which fields and methods to inherit from (B or C) due to ambiguity.

Solution is that Java does not allow multiple inheritance in class level, it can only come from the interfaces. Also the keyword default can be used on the method or the fields to help prevent ambiguity.

Is Java passing by value or passing by reference?

Java is always passing the argument by values.

Difference between Array and ArrayList.

ArrayList uses a dynamic array internally. However, the difference is that the array's size is fixed after initialization, and for the ArrayList the size can either expand or shrink down after a new element is inserted or deleted.

Difference between ArrayList and LinkedList What is immutable?

- ArrayList uses a dynamic array internally, LinkedList uses a doubly linked list
- ArrayList is slower in performance because when inserting or deleting an element from the Collection, it either needs to shift the element up or down, and a linked list does not need to do this
- ArrayList is good for storing and accessing. Memory in the heap is contiguous. Random access takes $O(1)$ time
- LinkedList is good for data manipulation. One element connects to another through reference. Random access takes $O(n)$ time.

Why is String immutable? How to create an immutable class?

String is immutable because of security reasons, caching, synchronization, class loading. For example: If there are multiple references pointing to a string, and one of them accidentally updates the string, it would cause chaos.

In Java, you can create an immutable by using the keyword final on both class level, method level, or field level. Final class prevents inheritance, final method prevents overriding, final variable/ field level prevents reassignment.

Difference between == and equals()

- == sign compares the memory address between the left side and the right side
- equals compares the value of the two objects that is being used for comparison. If either of the objects is null, it will throw a NullPointerException. Fixed by using Objects.equals().

Difference between String, StringBuilder, and StringBuffer

- String lives in the string pool. String Builder and String Buffer live in the heap memory
- String is immutable. StringBuilder and StringBuffer is mutable
- StringBuilder is not thread-safe. String and StringBuffer is
- StringBuffer is slower in performance, StringBuilder and String is faster

Difference between default and protected access modifier

- Protected: fields and method with this level of access modifier can be access via the subclass and same package
- Default: fields and method with this level of access modifier can only be accessed through same package only

How HashMap works internally?

- Hash map uses both Array and LinkedList structure internally
- Upon the initialization, HashMap has an array with a total of 16 buckets, with max capacity is 8 and a load factor of 0.75. Each bucket is a linked list and the nodes store the key- value pair.
- The index of each bucket to store the element is determined by the key and hashCode function.
- Within the same bucket, equals() is used to determined there is no duplicate on the key
- When the bucket reaches its max capacity, it will expand the array, and re-insert all the element.

How HashSet works internally?

- Hashset uses a map internally. Instead of key-value pair at each node, it is only 1 value per nodes. The index of which bucket to be stored in is determined by a the value and the hashCode function. the equals() is used to ensure that are is only 1 existing value, and no duplicate for teh same bucket.

Difference between HashMap, SynchronizedMap, and ConcurrentHashMap vs HashTable

- HashTable: java legacy, does not allow any null key or any any null values, thread-safe.
- the next 3 is java Morder:
 - + hashMap: allow 1 null key and multiple null values. NOT THREAD SAFE
 - + SynchronizedHashMap: Thread-safe. Performance is slow due to only 1 lock for the entire hashMap object.
 - + ConcurrentHashMap: There is 1 lock for each individual buckets. Multip[le threads can access to the same hashMap objects at the same time. However, there can only 1 thread per bucket at a time

Difference between List, Set, Map, Queue

- List is the interface that contains ordered element, allow duplicate
- Map is the interface that cotnains un-order element, store key-value pair, contains un ordered element. Allow dupliocate on the value but not the key
- Set is the interface that contains unorder element, and no dplicate is allowed
- Queue is the interface that follow the First in first out rule. Elements enter from the back and exist at the front. The order enter is the same order that exist.

the order here is reffering the insertion order

What are types of Exception?

there are 2 types of exceptions:

- Checked exceptions is referring to the error that happens in during the compiling time. Can be handled by a try-catch block. If it is not being handled. The compiler will throw coimpiling error. EX: FileNotFoundException

-Runtime Exceptions referring to the error that happens during the runtime. can be hadnled by a try-catch block. if it is not being handled, the compiler will not yell at you. It is up to the developer to determined what and how to deal with the exception. Ex: ArithmeticException. where you are trying to divide a number by 0;

How do you handle exceptions in Java?

In java, the simplest way to handle the exception is by using a try-catch block. If an exception is thrown inside the try block, it will jump to the catch block directly. The catching order matters. It needs to go from specific to broad. IF the broad exception is being caught first, it will be handled right away.

How do you handle exceptions in your web application?

In my web application, I handled all the exceptions in 1 place, known as a global exception handler. To do this, first you annotate the class where you want to use the to handle all the exception with `@ControllerAdvice`. Then each method will be used to handle a specific exception, and will be annotated with `@ExceptionHandler` with a specific class of exception.

Explain about Exception Propagation

When the exception is being thrown inside a method. it will travel up the stack until it is being handled. Also known as global exception handler.

How to create customized exceptions?

To create a customized exceptions, you can either extend it from `Exception` for a checked exception or `RuntimeException` for an unchecked exception. Either the cases, you will need to have the keyword "Super" inside the constructor of the customized exception class

Difference between final, finally, finalize

- Final is the keyword that is used to create constant/immutable
- Finally is the code block that goes along but right-after the try-catch block. This code block will be executed regardless whether or not an exception is being thrown
- Finalize is the method from the `Object` class that is called just right before the Garbage Collector collects the objects. From java 9+ this has been deprecated, and not recommended to use for cleanup

How to create a thread?

A thread can be created by extending from `Thread` class or implementing from `Runnable` interface. Either the cases, you still need to implement the body of the method, which is defining the task for the thread to run. The implementation of the method body can be done by using lambda function.

Difference between Runnable and Callable

- `Runnable` does not return a result when the task is done. `Callable` does. The result of `Callable` is held in either `Future` or `CompletableFuture`
- `Callable` can throw a checked exception
- `Runnable` overrides `run`, and `Callable` overrides `call`

Why wait() and notify() are in Object class but not in Thread Class

- Wait and notify is inside the `Object` class and not the `Thread` class because. If it is inside the `Thread` class, only that specific thread is locked, while the `Object` can still be accessible to multiple other threads at the same time.
- if it is in `Object` class, it ensures that there can only be 1 thread access to the `Object` at a time.

What is join()?

`Join()` signals the end of the life cycle of the thread. The main thread will remain blocked until the worker thread that the main thread called is dead.

Explain thread life cycle

There are 5 stages when it comes to a life cycle of a thread:

- New: this is a very first phase very a thread is newly created
- Ready / Runnable: This is the very where the thread is submitted to the scheduler. The Scheduler will run the thread whenever it is able to. Developer have no controll over whne the scheduelr run the thread. A thread can get into this stage by going from new, blocked, and running
- Running: this is the stage where the thread is acutally executing the task. From here, it can either go to terminated when the task is done, go back to runnable throguh yield, or block by calling wait() or sleep()
- Blocked: this is the stage where a thread is wait or blocked. When it is ready to back to resume the task it is working on via notify() from another thread, it will be submit to the scheduler and go to thje ready/runnable stage
- Terminated: this is the stage when the taske is done and thread is dead

Difference between sleep() and wait()

- wait() can only be existed inside he try-catch block, sleep doesn;t
- wait will release the lock if the thread is holding any, sleep doesnt
- sleep will automatically wake up after an interval of time and resume the task it is working on; the thread calling wait will remain blocked until another thread notify it.

What is countdownLatch?

- countdownLatch ensure that the worker threads does not execute until the main thread is done with its tasks.
- countdownLatch can be created by passing an integer to the constructor. That is the number of parties that is being invited to the event
- the main thread will remain blocked until the count number of parties reaches 0

CountdownLatch Vs CyclicBarrier

- CountdownLatch is event-batch. One thread waits for other threads to be done, and cannother be reused.
- CyclicBarrier is peer-to-peer synchronization. It is the Threads waiting for each other before continue, and is reusable.

What is semaphore?

Semaphore is ensure that there is a number of process that can access to an object at a time. Creat by passing a non-negative natural number to the constructor.

Difference between Future and CompletableFuture (+ 1 missing)

Advantage of executor service instead of just create a thread and run it

- Executor Service reduce the overhead of manually create thread.
- No need to handle the exception
- After the thread is done with its task, the executorService will put the thread back into the pool to be later use in the future

What is deadlock? How to handle deadlock?

DeadLock is a situation where there are 2 or more threads hold each other's lock causing the threads to wait for each other indefinitely.

To prevent a deadlock situation, you can put one thread to wait, the thread being put to wait will release the lock, and the dependency of threads waiting on each other will be broken. Another way is by using ReentrantLock with tryLock + timeout, or reduce the scope of lock, or avoid nested lock.

What is synchronization? How do you do synchronization?

Synchronization ensures that there is only 1 thread that can have access to read/write/update a critical code block at a time. This can be done by using the keyword "synchronized" on either the method or directly on top of the critical code block with the className.

What is readWriteLock?

readWriteLock ensures that multiple threads can read an Object but there can only be 1 thread write to the object at a time.

What is a marker interface? Also provide an example and explain how it works

marker interface is the interface without any fields or methods. It is used for the compiler and JVM during the compiling time, so the compiler and JVM have extra context about the text that implements this interface.

What does it mean by Serialization?

Serialization means writing the object to files or an array of bytecode for transmission, which can be used to construct back into an original object through deserialization.

What's the purpose of transient variables?

transient variables are the variables that are skipped during the serialization process. Some of the variables that should not be serializable are username, password, or an untrusted Object.

How do you do the serialization and deserialization? (+1 missing)**What is memory leak?**

Memory leak is when an unused object is occupying an unnecessary space in memory. When the Java Garbage Collector is trying to remove it, it cannot do so because there are still references to it. An example for this is developer forgot to close connection to other service.

What are different types of memory?

There are 5 different types in memory:

- Heap
- Stack
- Metaspace
- PC Registration
- Native Method

Heap vs Stack memory?

- Heap is the main memory area where everything lives
- Stack memory is the area that is locally when method call upon each other, that stores methods calls, local variables, primitive variables, references to objects

equals() & hashCode() contract

Two Objects have the same hashCode() does not guarantee to be equals() but if equals() it is guarantee to have the same hashCode()

Comparable vs Comparator

- Comparable define natural ordering, implemented inside the class by overriding compareTo and only 1 sorting rule per class
- Comparator define custom sorting rule, implement outside of the class with compare() method, and allow multiple sorting strategy

Volatile keyword

Volatile is the keyword that ensure the variable is being read from the main memory, and not from local cache. It ensure that all the threads always see the most recent value. This guarantee visibility but not guarantee atomicity

Atomic classes(miss)

What is ReentrantLock?

ReentrantLock is from another class in Java. It also provided synchronized mechanism but more flexible due to feature like tryLock with timeout, fairness policy. Same thread can acquire same lock multiple times.

Java 8:

Have you used Java 8? What features are you familiar with?

- Stream API
- Functional Interface
- Lambda function
- Optional Class

What is the lambda expression?

- Lambda function is intended for short use and quick function without any names. It also takes in parameter and return a result

What is the functional Interface? Follow Up: Is @FunctionalInterface annotation required?

Functional interface is the interface that have exactly 1 abstract class but it may have multiple static or default method. It is not required to have the annotation @FunctionalInterface for the interface but it is a good practice to do so. When using the functional interface, the body of the method needs to be implement, and can be done by using lambda expression

How lambda expression and functional interface works together? Also provide your own example

Because functional interface have exact 1 abstract class. When lambda expression is being used to the implement the body of the method, it knows that the implementation is for the abstract class.

```
@FunctionalInterface
public interface Number{
    Integer square(Integer x);
}
Number x = (a)->{return a*a;};
System.out.println(x.square(3));
```

(Please use Java 8 stream api to answer this question) We have an employee class which has id, name and age. Find out the employee name, start with "a", and convert the list of employees to the map with the key is the employee id and the value is the employee object itself.

```
someListName.stream()
    .filter(x -> x.getName().startsWith("a"))
    .collect(Collectors.toMap(
        x -> x.getId(),
        x -> x
    ))
)
```

(Please use Java 8 stream api to answer this question) Given a list of Strings, "aa", "ab", "bc", null, "abc". Find out all the String start with a and sort the result and return a list of String.

```
someListName.stream()
    .filter(x -> x.startsWith("a"))
    .collect(Collector.toList())
```

What is the default method and what is the static method?

default method is a method from the interface with a pre-defined body. When the class implement the interface, it can choose to override the method body but it is not mandatory.

static method is the method that is pre-compiled during the compiling this. it does not need to initiate an instance to use it

What is Optional in Java 8?

Optional in Java is an object that contains non-null objects. If the object is present inside the container, it will return true upon calling isPresent(); Otherwise, it will return false.

What is a Stream in Java 8?

Stream in Java is defined as a sequence to the datasource. the datasource here is referring to elements from Collections, array, list, set that provided the data to the stream. There are 2 types of operations that can be performed on a stream: Terminal operations and intermediate operation.

- Intermediate operation: filter, map, sorted
- Terminal operation: collect, forEach, min, max

What is Parallel Stream?

Parallel in stream divide a workload across multiple different thread so that the work can be done faster

Difference between Stream and Collection?

Stream does not hold data, it operate on the data Source like Collection

Collection hold data with different datastructure like Set, List, Queue, Map

Stream is traversable and consumable,

Collection is traversable but not consumable

What is the difference between intermediate and terminal operations?

- Intermediate operations return stream-values, it does not modify the data from the original datasource
- Terminal operations return non-stream values like primitive data, list, set, collection, array, or it may not return anything. It also can modify the data from the original data source

What is lazy evaluation in streams?

lazy evaluation in stream is that the intermediate operations does not gets executed until the terminal operations is invoked

What is the difference between map() and flatMap()?

Different between Predicate, Function, Consumer, Supplier

- Predicate : takes in parameter and return boolean. Used for testing condition
- Function: takes in parameter and return a result. Used for transformming and mapping
- Consumer: takes in parameter and does not return anything. Used for performing an action
- Supplier: does not take in anything and return a result. Used for supplying value

What happens if you reuse a stream after a terminal operation?

You cannot reuse a stream after terminal operation. If doing so, it will throw IllegalStateException

What is the difference between findFirst() and findAny()?

- findFirst() is used in sequential stream, it will return the first element based on the encounter order. Garentee the return value

-findAny() is used in parallel stream, it will return the first element of whichever the first thread sees. Does not garentee the return value

What is the difference between limit() and skip()?

- limit(): only take the first N elements
- skip(): skip the first N elements

Why was Optional introduced?

Optional was introduced to avoid dealing with NullPointerException

Difference between orElse() and orElseGet()?

Java 8 Extras (GOOD TO KNOW BUT NOT REQUIRED)

What is reduce () ? Give a practical example.

How does `distinct()` work internally?

How does `Collectors.groupingBy()` work?

What is the difference between `groupingBy()` and `partitioningBy()`?

How do you count elements using streams?

How do you convert a list to a map using streams?

What happens if duplicate keys exist in `Collectors.toMap()`?

How do you handle duplicate keys in `toMap()`?

How do you join strings using streams?

What is the difference between `map()` and `flatMap()` in `Optional`?

Why should `Optional` not be used in fields or method parameters?

What happens if you call `get()` on an empty `Optional`?

How does `Optional` help prevent `NullPointerException`?

What is a parallel stream?

How does parallel stream work internally?

When should you NOT use parallel streams?

Are streams thread-safe?

What problems can occur with parallel streams?

What is `ForkJoinPool`?

What is the difference between `Future` and `CompletableFuture`?

What is the difference between `thenApply()` and `thenCompose()`?

What is the difference between `thenApply()` and `thenAccept()`?

What does `thenCombine()` do?

How do you handle exceptions in `CompletableFuture`?

What is the difference between `supplyAsync()` and `runAsync()`?

Why was the new Date API introduced in Java 8?

Difference between `LocalDate`, `LocalDateTime`, and `Instant`?

Is the new Date API mutable?

What is `ZoneId`?

How do you convert between `Date` and `LocalDateTime`?

What is a default method?

What is a static method in interface?

What happens if two interfaces provide the same default method?

Can you override a default method?

Design Patterns

54. Design Pattern that you have used

- Proxy Design Pattern
- Singleton Design Pattern
- Observer Design Pattern
- Factory Design Pattern
- Builder Design Pattern

55. What is the Singleton Design Pattern?

Singleton Design Pattern ensure that there is only 1 instance of the class exist throughout the application. If the instance is already exist, it will simply return the instance. If it does not, then create a new one, then return it.

The singleton object will continue to exist until the application is shut down or destroy method is called.

56. How to create a singleton? (eager initialization and lazy initialization) Provide code

Eagerly initialization: The instance is created at class loading time

```
Public Singleton{
    private static final Singleton instance = new Singleton();
    private Singleton(){}
    public Singleton getInstance(){
        return instance
    }
}
```

Lazy initialization: the instance is created the first time a thread is trying to get the instance:

```
Public Singleton{
    private static volatile Singleton instance
    private Singleton(){}
    public Singleton getInstance(){
        if(instance == null){
            synchronized(Singleton.class){
                if(instance = null){
                    instance = new Singleton();
                }
            }
        }
        return instance
    }
}
```

57. Is Singleton thread safe? 58. How to make singleton thread safe? (for both eager and lazy)

Singleton is thread-safe depends on the implementation:

- If the implement is eagerly initialization: then the singleton is inheritedly thread-safe through class loading
- If the implementation is lazy loading, can make it thread safe by using keyword volatile, synchronized, and double lock null checking

59. How to prevent Singleton Pattern from Reflection, Serialization and Cloning?

- Cloning: Override clone() method and throw CloneNotSupportedException;
- Reflection: Best practice is by using enum;
- Serialization: implement readResolve();

60. What is the factory design pattern? Why do we use factories? How do you use this design pattern in your application?

61. Provide a code example for factory design pattern

62. Difference between factory vs. abstract factory design pattern

63. Provide a code example for builder design pattern

Example given a class:

```
Class User{  
    private String Username;  
    private String Password;  
    private Integer SSN;  
}
```

```
Class User{  
    private String Username;  
    private String Password;  
    private Integer SSN;  
  
    private User(Builder build){  
        this.username = build.username;  
        this.password = build.password;  
        this.SSN = build.SSN;  
    }  
  
    public static class Builder {  
        private String Username;  
        private String Password;  
        private Integer SSN;  
  
        public Builder Username (String username){  
            this.username = username;  
        }  
  
        public Builder Password (String password){  
            this. password = password;  
        }  
  
        public Builder SSN (String SSN){  
            this. SSN = SSN;  
        }  
  
        public User Build(){  
            return new User(this);  
        }  
    }  
}
```

64. Explain proxy design pattern

Proxy design pattern is when you don't call to the real object directly. Instead, you call to the Proxy class that wraps around the real object. The proxy can do extra work like security check, logging, caching, lazy loading. The call then will be forwarded to the real object internally.

65. Explain Observer design pattern

Observer design pattern is when an object keeps a list of objects that are interested in it. When the state changes, it will notify all of them. This is useful for event-system, notification where multiple objects need to react to a change.

66. Explain Chain of Responsibility design pattern

Chain of Responsibility is when a request is being passed through a chain of handlers. Each handler can decide whether to handle the request or pass it to the next one. This avoids tight coupling between the sender and the receiver.

JDBC & Hibernate & Database

67. What is SQL Injection? How to solve it?

SQL Injection is the query that is always correct, which can return all the data from the database. There are many ways to solve SQL Injection, one of which is by using PreparedStatement. It is precompiled with placeholder and user input is binded separately. The input is treated as data, and not as executable SQL queries.

68. Difference between Statement and PreparedStatement

Statement:

- Execute Raw query
- Parse + Compiler every query execution
- Vulnerable to SQL Injection
- Slow for repeated query
- Simple for static, one-time query
- No binding parameter

PreparedStatement:

- Precompiled with placeholder
- Prevent SQL Injection
- faster for repeated queries
- Support batch update

69. How to handle transactions manually in JDBC?

- By default, transaction in JDBC automatically commit after every query transaction. We can turn this off, any by doing so, the developer needs to manually do the commit in order for the changes to reflect on the database. We want to do this because we may want to bundle multiple operations together into 1 transaction and still follow the ACID property. In the event of failure, we can roll back to the previous successful state.

70. What is ORM and why need ORM?

ORM stands for Object Relational Mapping. It helps automatically generate query to manipulate the data in the database. It helps transfer the data from the database into the mapped object. We need it because it provides a dynamic way to connect to multiple different databases. Later on, if we decided to change the tech stack, we can simply change the configuration and without changing the existing source code.

71. What is Hibernate?

Hibernate is an ORM tool that maps Java classes to database tables in Java Programming Language. By using Hibernate, a developer can focus on handling the data within the instance of the mapped class rather than the data from rows and columns in the database. Because it is an ORM tool, it helps generate queries when there are changes to the instance of the mapped object.

72. How to configure Hibernate?

All the configuration of Hibernate is being stored inside `Hibernate.cfg.xml`. In here, we can find settings for:

- Driver class name
- URL to the database
- username
- password
- Hibernate Dialect based on the database being used
- `hb2ddl.auto`
- Mapping of the class to database table

73. What is Session and SessionFactory in Hibernate?

Session is a connection to the database. Backend Application uses session to connect and query for data from the database.

SessionFactory is a place which stores new and used sessions after query for data from the database. It reduces the overhead by reusing the existing session rather than creating one from ground up, which can be expensive. It follows pool design which has maximum number of connections, minimum number of connections, number of idle connections, and idle wait time.

74. Is session in hibernate thread safe?

Session in hibernate is not thread safe. It means to represent 1 unit of work - usually a request. It maintains states like first level cache, changes to an object. By sharing the session across multiple threads, it can cause data inconsistency or unexpected behavior. Each thread should have its own session.

SessionFactory is thread safe. Created once and can be shared among the threads (Singleton)

75. Difference between `getCurrentSession` vs. `OpenSession`

- `getCurrentSession` will grab an existing session from the session factory, use it, then return it to the pool once done. Developer does not need to close the session afterward.

- `OpenSession` will create a new session from ground up, which can be expensive. Once done using, developer needs to manually close the session; Otherwise, it will cause memory leak.

76. Difference between `get()` vs. `load()`

- Get will fully initialize the object in the heap area. Use if you are unsure whether or not the Object exist in the database
- Load will return a pointer to the object that is specified. It is slightly faster than get. Only use if you are sure that the object exist in the database; Otherwise, it will throw a `ObjectNotFoundException`

77. Difference between update, merge, saveOrUpdate

78. How to do Many-to-Many mapping in hibernate

79. Give an example with @ManyToOne and @OneToMany

80. Fetching strategy in Hibernate

In Hibernate, fetching strategy determines when and how the related data is being loaded from the database into the main memory. There are 2 types of fetching:

- Eagerly loading: related data is being loaded immediately after the main object is loaded
- Lazy loading: related data is being loaded only when it is actually being used

81. What is lazy fetching?

Lazy fetching in hibernate is a fetching strategy that only loads the related data when it is actually being used. It saves the memory and improves the performance by not loading the unnecessary data upfront.

82. What is LazyInitializationException?

`LazyInitializationException` happens when the lazy-object is being accessed when the session containing the main object is closed.

83. How to prevent LazyInitializationException?

- Keep the session open while accessing the lazy object
- Eagerly load all the data into the main memory
- Use a DTO or service layer to load in all the data that you know you will need

Basically not trying to access the lazy-object when the session containing the main object is closed

84. Why do we prefer Hibernate over JDBC?

- Caching and performance: Hibernate has a built-in cache that improves the performance
- Less Boilerplate: you don't need to write raw queries, hibernate generates it for you
- Object oriented: work with Java objects, instead of rows and columns from the database
- Auto mapping: Java classes are automatically mapped to database tables
- Database portability: ability to switch to different databases without changing the existing code

85. What is HQL and What is Criteria? What is type safe?

- HQL: Hibernate Query Language: it is a way to ask the database for data using Java Objects
- Criteria: it is a way to build queries step-by-step using java logic. Use when queries change dynamically
- type safe: system checks during the compiling time to ensure that you are using the right kind of data before the system runs

86. Caching in Hibernate

Caching in hibernate is a way to store data temporarily without hitting the database everytime you need data with the same information. There are 3 levels of caching in hibernate:

- 1st level cache: built-in per session. Hibernate automatically keeps objects of the object being loaded into the main memory. No extra setup is needed
- 2nd level cache: shared across multiple sessions. It stores data and states to reduce database hits
- Query cache: it caches the result of a specific query; So repeated queries can be served faster

87. Explain ACID

ACID is a set of rules that ensure the database is correct and reliable even when something goes wrong:

- A - Atomicity: Transaction happens either completely or not at all
- C - Consistency: Database always stays in a valid state. Rules and constraints are never broken
- I - Isolation: Multiple transactions happen simultaneously and do not affect each other
- D - Durability: Once a transaction is complete, it is permanent; Even in the event of a crisis like power outage or app crash

88. SQL: Orders table has 4 columns CustomerId | City | Sales | ProductID. Write a query to find the cities which have total sales greater than 1 million.

```
Select city, SUM(Sales) as total_sales
from Orders
group by city
having Sum(sales) > 1000000
```

89. Difference between SQL and NoSQL

90. What is inner join, left join, right join?

- Inner join returns all rows with matching columns in both tables
- Left join will return all rows from the left table, and the matching rows in the right table. If there is no matching row in the right table, it will return null
- Right join will return all rows from the right table and the matching rows in the left table. If there is no matching row in the left table, it will return null.

91. Why do we prefer "From Table A inner join Table B on A.id = B.id." instead of "From Table A, Table B where A.id = B.id"?

92. Explain CAP

CAP theorem says that in a distributed system, it is guaranteed there is always 2 out of 3:

- C: Consistency: all reads get most recent write (all nodes see the same data)
- A: Availability: All requests will get a response, even if it fails
- P: Partition Tolerance: System still keeps running even if the network splits and nodes are not talking to each other

93. Is MongoDB CP or AP?

94. When do you choose to use Relational DB and when to use Non-Relational DB?

Spring

95. Difference between Spring and SpringBoot

- Spring: framework to build apps. You must manually configure a lot (XML/Java Config, dependencies, server setup)
- Spring Boot: Built on top of Spring, provides auto-configuration, starter dependencies, embedded Tomcat server, production features like Actuator, so you can run the app with minimal configuration.

96. What is dependency injection? Why need dependency injection? How does spring IOC container work?

97. How to inject beans in spring?

In Spring, there are 3 different way to injects been

- Constructor injection (Preferred/ Recommend)
- Field injection (no recommend)
- setter injection

98. How to configure/create beans in spring?

- Component Scan: @Component, @Controller, @Service, @Repository
- Configuration to other services: @Configuration, @Bean
- XML for older version of Spring

99. Difference between BeanFactory and ApplicationContext?

- Bean Factory: Basic container, lazy initialization by default, minimal feature
- Application Context: Advance container, support events, AOP, and auto bean post processor, and eager initialization by default

100. Describe bean scopes that supported by spring ? How to define the scope of a spring bean?

Spring support 6 different scopes:

- By default, Singleton - not thread safe
- web scopes: request, session, application
- prototype, and global session

There are 2 main ways to define a scope of spring bean:

- By using scope annotation with a specific scope name level
- through XML configuration for older Spring version

101. Spring bean lifecycle?

Spring manage bean life cycle from creation to destroy inside Application Context - IOC Container:

- Container setup: Spring create an Application Context (IOC container) and loads all beans definition onto it
- Spring create the instance of the beans through constructor and factory methods. At this point, the bean is still not fully initated yet
- Spring inject the dependency through constructor, fields, or setter
- If the class implement the aware interface, it will call to the aware callback
- postProcessBefore initialization: this step allow modification of the bean before the initialization logic runs
- Initialization logic
- postProcessAfter initialization logic. This is where AOP proxies usually are created
- beans are now in the container, can be injected and to be used by other beans
- destruction phase: when the app shuts down, it will call to @PreDestroy. this is where you can up the resources.

102. What's your understanding on @autowired? How spring know which bean to inject

@Autowired is used for dependency injection. It tells Spring to inject a matching bean from the application context to the place that is annotated. Can be placed on field, constructor, or setter methods

For Spring to know which bean to inject:

- by type: Spring first looks for a bean in the context that matches the field/parameter type.
- by name: if there are multiple beans with same type, it will check the variable name or parameter name to match the bean ID
- with Qualifier: If there are multiple beans exist and you want to specify exactly which one, use Qualifier("beanName")
- primary bean: you can mark a bean as @primary to tell Spring to prefer it if there are multiple candidates exist

103. What is circular dependency

Circular dependency is when there are two or more depend on each other directly or indirectly, which creates a cycle. A-> B-> C-> D-> A. One solution to resolve this is by using @Lazy, which delays the initialization of one bean until it is actually used

104. What's the usage of @Qualifier

The @Qualifier annotation in Spring is used to resolve ambiguity when multiple beans of the same type exist in the context. It tells Spring exactly which bean to inject.

105. What is the usage of @SpringBootApplication

The @SpringBootApplication annotation is the main entry point for a Spring Boot application. It's essentially a convenience annotation that combines three key annotations into one:

- SpringbootConfiguration
- EnableAutoConfiguration
- ComponentScan

It reduces boilerplate, centralizes entry point, and convention over configuration

106. How you handle transaction in Spring Boot

In Spring Boot, transactions are handled mainly through declarative transaction management using the @Transactional annotation. You can annotate a class or a method that requires a transaction, and Spring automatically manages the transaction lifecycle: it begins the transaction when the method starts, commits it if everything completes successfully, and rolls it back if an exception occurs.

By default, Spring rolls back transactions on unchecked exceptions (runtime exceptions), but you can configure it to roll back for checked exceptions as well. Spring Boot auto-configures the appropriate transaction manager based on the database or JPA implementation, so developers don't need to manually start or commit transactions.

This allows multiple database operations within a method to be executed as a single atomic unit, ensuring consistency, rollback on failure, and simplifying transaction handling in the application.

107. Describe SpringMVC (Need to mention DispatcherServlet)

Spring MVC is a framework to build web applications following the Model-View-Controller (MVC) pattern. It separates responsibilities so that:

- **Model** – represents the data and business logic.
- **View** – renders the response (HTML, JSON, XML, etc.) to the client.
- **Controller** – handles HTTP requests, processes input, and returns a response.

At the heart of Spring MVC is the **DispatcherServlet**, which acts as the **front controller**:

1. It receives all incoming HTTP requests.
2. It consults **HandlerMapping** to find the appropriate controller for the request.
3. It invokes the controller method to process the request.
4. The controller returns a **ModelAndView** (model data + view name).
5. DispatcherServlet then resolves the view (via **ViewResolver**) and renders the response.

108. Difference between @Controller and @RestController

- @Controller

- Used for Spring MVC controllers that return **views** (like JSP, Thymeleaf)
- Methods return **view names** by default
- Does **not** automatically serialize return values to JSON/XML
- Use case: Server-side rendered web pages

- @RestController

- Used for REST APIs that return **data directly** (JSON, XML)
- Methods return **response body** directly
- Combines @Controller + @ResponseBody implicitly
- Use case: RESTful services / APIs

109. @RequestBody vs. @ResponseBody

- @RequestBody

- Binds **HTTP request body** to a Java object
- Automatically deserializes **JSON/XML** → **Java object**
- Used in **controller method parameters**
- Common for **POST/PUT requests**

- @ResponseBody

- Converts **Java object** → **HTTP response body**
- Automatically serializes to **JSON/XML**
- Used in **controller method return types**
- Common for **GET/POST responses**

110. @PathVariable vs. @RequestParam. Provide your own example for each. Write the endpoints and the urls that you can invoke these endpoints

111. How to make your endpoint consume xml and produce json?

- Use **consumes** to specify the expected request content type (e.g., XML)
- Use **produces** to specify the response content type (e.g., JSON)
- Ensure **Jackson (or similar library)** is on the classpath to handle JSON serialization
- Ensure **JAXB (or similar library)** is available to handle XML deserialization
- Typically used in **@PostMapping, @PutMapping, or @RequestMapping**

112. How do you use Spring Repository?

113. What are different kinds of http methods

- Follow up: Difference between post vs. put

Http methods include: Get, Post, Put, Patch, Delete, Options.

The different between Put and Post is that:

- post will create a new resource, non-idempotent
- put updates an existing resource, idempotent.

idempotent means multiple calls and the same result

114. Difference between SOAP vs. REST

SOAP: Simple Object Access protocol

- Protocol-based (strict standards)
- Uses **XML** for messages
- Built-in **error handling** and **security (WS-Security)**
- Stateful by default, can support transactions
- Requires **WSDL** for service contract
- Slower due to **XML parsing** and verbosity

REST: (Representational State Transfer)

- Architectural style (not a protocol)
- Uses **JSON, XML, or other formats**
- Lightweight, faster performance
- Stateless by default
- Uses standard **HTTP methods** (GET, POST, PUT, DELETE)
- Easy to consume by browsers and mobile apps

115. what are the principle of REST? Is REST stateless?

Principles of REST - memory trick: CSCULRC

1. **Client-Server** → separation of concerns; client handles UI, server handles data
2. **Stateless** → each request contains all information; server does not store client state
3. **Cacheable** → responses must indicate if they are cacheable to improve performance
4. **Uniform Interface** → consistent way to access resources (resource identification, representations, standard methods)
5. **Layered System** → client cannot tell whether it is connected directly to server or through intermediaries
6. **Code on Demand (optional)** → server can extend client functionality by sending executable code (e.g., JavaScript)
7. **Resource-Based** → resources identified by URIs
 - yes, REST is stateless. it is the second principle out of the 7th principle guideline from REST

116. How to maintain user logged in using REST service

Since REST is stateless, we maintain login using JWT tokens. When a user signs in, the backend generates a JWT with expiration and sends it to the client, usually stored in a cookie. The client sends this token with each request, and the server validates it to keep the user authenticated.

117. What is Spring AOP?

AOP stands for Aspect Oriented Programmin, it Separates **cross-cutting concerns** (like logging, security, transactions) from business logic. it uses aspects to defeine the task/ code to run, advice to indicate when to run the aspect, and join point to determin where the aspect is applied to, and join point to specify where the advice should run

118. Talk about Spring Security

Spring Security is a framework for **authentication and authorization** in Spring applications. It handles login, role-based access, password encoding, and protects against common security attacks like CSRF. It works by intercepting requests and verifying credentials before allowing access.

119. What do you use for testing? (Mockito/JUNIT)

I use **JUnit** for unit testing and **Mockito** for mocking dependencies. JUnit helps verify method behavior, and Mockito allows me to simulate objects and test components in isolation.

120. Talk about the design patterns in the spring framework

Spring heavily uses **design patterns** to keep the framework modular and flexible. Key ones include:

- **Singleton** → beans are singletons by default
- **Factory** → BeanFactory creates and manages beans
- **Proxy** → for AOP (method interception)
- **Observer** → for event handling and application events

These patterns help Spring manage **dependency injection, cross-cutting concerns, and reusable components** efficiently.”

Microservice

121. Difference between Monolithic vs. Microservice

122. What is the benefit of microservices rather than monolithic application? Is the microservice always better than monolithic application?

123. What is cascading failure? How to prevent this failure?
124. What is fault tolerance? How to make your microservice fault tolerance?
125. How to communicate between microservices?
126. What is Swagger?
127. How do you monitor your application?
128. Why do we need a gateway?
129. How many environments does your application have?
130. How to deploy your application?
131. Usage of Jenkins.
132. Describe your microservice architecture(candidates were asked to draw the architecture during the interview)
133. How to debug your microservice? (In other words, how to do the troubleshooting when there is an error in the microservice application?)
134. How do you implement async in web application?
135. What is the Message queue? RabbitMQ is based on what protocol? How do you use RabbitMQ?
136. How to secure your endpoint? (In other words, How can you check if http call is valid in microservices?)
137. Where do you store your configuration file when you use microservices?
138. How did you do user authorization in microservices
139. Vertical Scaling and Horizontal scaling in your application
140. Talk about maven. What methods you used in maven
141. Tell me about your experience with Cloud Service. Ex. AWS, GCP, Azure
142. What is Kafka?
144. How do you do version control?
145. How do you do production support? (e.g., Jira)